SmartBin Route (Trash Track): Project Documentation

QNX eHACK 2026 | Problem Statement SL37 | Team Deadline Drivers

1. Overview

SmartBin Route is a real-time waste collection scheduler built on the QNX RTOS. It monitors the fill level of waste bins, builds an adaptive collection route, and immediately overrides that route when a bin reaches the overflow threshold. A sensor-controlled bin door (level indicator sensor plus ultrasonic sensor) keeps a full bin closed until the collection vehicle clears it.

2. Problem Statement (SL37)

Intelligent Waste Collection Scheduler: optimize collection routes based on dynamic bin status and vehicle availability, while emergency and overflow conditions override normal route optimization.

Scenario. Bins fill at different, unpredictable rates, but fixed routes treat every bin alike. Trucks waste fuel on near-empty bins while full bins wait.

Criticality. An overflowing bin cannot wait for the next periodic route update, so emergency handling must take priority over normal scheduling.

Requirements.

- Continuously monitor bin status.
- Generate an adaptive route from that status.
- Trigger an immediate route override when fill reaches 90%.

Success metric. Overflow event, then Emergency Task, then override pulse, then route recomputation.

3. Proposed Solution

The solution has three cooperating parts.

1. **Bin monitoring.** Each bin's fill level is tracked continuously and stored in a shared bin status map.
2. **Adaptive routing with emergency override.** A scheduler ranks bins by a weighted score every 5 seconds. A dedicated high-priority Emergency Task watches every reading and forces an immediate re-route when a bin reaches 90%.
3. **Smart bin door.** Two sensors decide whether the door may open (see section 4).

4. Sensors and Smart Door

The uploaded baseline code simulates bin fill levels in software. The door and sensor logic below is the hardware-facing part of the design and is the planned extension of that baseline.

4.1 Ultrasonic sensor

It sends a short 40 kHz sound burst and measures the time until the echo returns. Distance = (echo time x speed of sound) / 2, using about 343 m/s for sound; the division by 2 accounts for the round trip. A typical module such as the HC-SR04 reads about 2 cm to 4 m. In this design it checks for free space above the bin opening.

Why it is used: it is non-contact (hygienic), cheap, and gives a direct distance reading. Limitations: soft or angled surfaces scatter the echo, temperature slightly changes the speed of sound, and uneven trash piles can mislead it.

4.2 Level indicator sensor

It measures how full the bin is (for example with a float switch, capacitive probe or weight sensor) and reports a fill percentage. That value feeds the bin status map and the 90% overflow check. It is independent of surface shape, so it covers the ultrasonic sensor's weak spots.

4.3 Door behaviour

- **Open:** there is space above the bin and the fill level is below the threshold.
- **Locked:** no space, or fill is at or above the threshold. The bin is flagged as overflowed.
- **Reset:** the door stays closed until the vehicle clears the disposal. After collection the fill level resets to 0 and the door can open again.

5. System Architecture

The application is one QNX process with several POSIX threads, one per independent real-time job. Threads communicate by QNX pulses and share data structures protected by mutexes.

Item	Value
Bins (streets)	4 simulated bins
Overflow threshold	90% fill
Bin update period	1 s
Normal route recalculation	every 5 s
Vehicle cycle	every 3 s
Logging	console plus log file

Data flow: Bin Monitor threads send a BIN_READING_PULSE to the Emergency Task. If a bin is at or above 90%, the Emergency Task sets its overflow flag and sends an OVERRIDE_PULSE to the Route Scheduler, which recomputes the route immediately. The Vehicle Task services the first bin in the route and resets it.

6. Threads

Thread	Priority	Responsibility
Emergency Task	25	Receives each bin reading, checks the 90% threshold, marks overflow, sends the override pulse, logs the response time.
Route Scheduler	20	Owns a channel, runs a 5 s timer, recomputes the route on a timer pulse or an override pulse.
Bin Monitor (x4)	15	One per street. Every second it updates fill level (capped at 100%) and sends a reading pulse.
Vehicle Task	10	Every 3 s services the first bin in the route, then resets its fill, overflow flag and wait time.

The main thread initializes the bins, writes a startup banner, creates the threads (Emergency and Route Scheduler first, a short pause so their channels exist, then Bin Monitors and Vehicle) and waits on the Emergency Task.

Why threads. Threads share the bin map directly and are cheap to create and switch. Separate processes would force all shared data through IPC, which is too heavy here. A single polling loop would let an overflow wait behind slower work.

7. Scheduling

7.1 Thread scheduling

All threads use SCHED_FIFO with explicit priorities: Emergency 25 > Route 20 > Bin Monitor 15 > Vehicle 10. The highest-priority ready thread runs until it blocks or a higher one becomes ready, so the Emergency Task always preempts routing work.

- SCHED_RR was not needed because the threads mostly sleep or wait rather than compete at equal priority.
- Default (non-real-time) scheduling gives no latency bound, so it cannot support the emergency-response goal.

7.2 Route ordering

Score = 0.6 x fill + 0.3 x (1 / distance) + 0.1 x wait time. A bin flagged as overflowed always goes first; the rest are sorted by score. The scheduler copies the bin map under its lock into a local snapshot, sorts that copy with qsort(), and writes the result to the route array, so the shared data is not locked during the calculation.

A weighted score is cheap and easy to explain. A full optimal route search (travelling salesman) would cost far more than a handful of bins justifies.

8. Synchronization

Mutex	Protects
bin_map_mutex	Fill level, overflow flag and wait time shared by the monitors, emergency task, scheduler and vehicle.
route_mutex	The route array written by the scheduler and read by the vehicle.

Mutex	Protects
log_mutex	Console and log file output, so log lines from different threads never interleave.

Additional protection: the overflow flag prevents the same overflow from being handled repeatedly until the bin is serviced and reset.

Why mutexes. A bin update changes several fields together, so one thread must own the data while it writes. Semaphores are meant for counting or signalling, which pulses already provide. Spinlocks waste CPU while waiting. Atomics suit a single integer, not a multi-field struct.

9. IPC Interfaces

Mechanism	Used for
Channel + MsgReceive	The Emergency Task and Route Scheduler each own a channel and block on MsgReceive until a pulse arrives.
Connection + MsgSendPulse	Threads attach to a channel and send small asynchronous pulses.
BIN_READING_PULSE	Bin Monitor to Emergency Task; carries the street id.
OVERRIDE_PULSE	Emergency Task to Route Scheduler; forces an immediate route recompute.
ROUTE_TIMER_PULSE	A periodic 5 s timer delivers this pulse to the Route Scheduler.
Shared memory (in-process)	The bin map and route array, guarded by the mutexes above.

Why pulses. They are asynchronous, so a sender never blocks; they carry a small value; and QNX timers deliver them natively. Synchronous send/reply would stall the Bin Monitors for a reply nobody needs. POSIX message queues work but are heavier than native pulses. Shared memory alone cannot wake a sleeping receiver, so pulses signal events while shared memory holds the data.

10. Resilience and Emergency Override

Stress scenario. A bin crosses 90% during normal route operation.

Mitigation. The Emergency Task runs at the highest priority and receives bin readings independently of the 5 s route timer.

Recovery path. Emergency Task, then OVERRIDE_PULSE, then Route Scheduler, then an immediate recompute_route().

Protection. The overflow flag prevents repeated handling of the same overflow.

11. Verification and Demonstration

- Live demo: four simulated bin fill levels change and the route order updates, with an immediate re-route on overflow.
- The Emergency Task logs the response time of each override.
- Evidence to attach before submission: the QNX Momentics System Profiler screenshot showing CPU usage and thread timing.

12. Known Limitations

- The current baseline has no watchdog timer and no hard worst-case latency guarantee; both are validation and future work.
- Bins and the vehicle are simulated; there is no GPS or road distance data.
- The mutexes use default settings, so priority inversion is possible if a low-priority thread holds a lock that the Emergency Task needs. Enabling priority inheritance on the mutexes is the fix.
- The Vehicle Task logs while holding the bin lock; logging after unlocking would shorten the lock time.

13. Future Scope

- Integrate real ultrasonic and level sensors with the door lock hardware.
- Add GPS, road-distance data and multiple collection vehicles.
- Add watchdog and fault recovery, and formally measure bounded overflow-response latency.
- Add a web dashboard for operators.

Key learning. The main challenge was coordinating periodic scheduling, shared data and emergency events without race conditions.

Closing vision. A real-time collection system that spends less effort on low-priority bins and reacts immediately to critical overflow conditions.